

GenAR — AI Engineering Challenge

Build an AI System for Regulatory Safety Reporting

Use whatever you like — Claude, GPT, LangGraph, LangChain, LlamaIndex, MCP, raw API calls, plain Python. We're not scoring tool count. We're scoring **how you think about and design an AI system for a domain you've never worked in.**

The Problem

Pharmaceutical companies constantly receive reports of adverse events — a patient took a product, something happened, someone recorded it. Periodically, all of that has to be turned into a structured safety report for regulators.

For this exercise, you'll work with a real-format safety dataset for **Bisoprolol** (a common beta-blocker) covering **1,000+ cases over a one-year reporting period**. Each case has fields like age, sex, country, reaction, seriousness, outcome, and report date. Your job is to turn that raw pile of cases into a short, structured, **evidence-backed** report — a simplified version of what's called a **PADER**.

The one rule that matters more than any framework choice: **the report can only say what the data supports**. If the numbers say 1,024 cases, the report can say 1,024 cases. It can't say "no safety concerns were identified" unless something in your system actually establishes that.

You don't need prior pharma experience. We'll send a short primer alongside the dataset covering the handful of domain terms you need.

What You'll Receive

- `Bisoprolol_icsr_sample_1068rows.csv` — 1,068 case-safety report rows (patient demographics, product, reaction, seriousness, outcome, dates) covering a one-year reporting period
- `PADER_Starter_Guide.md` (also provided as PDF/Word) — the guide for this exercise: what a PADER covers, the section-by-section content expected, and the official regulatory references it's adapted from
- `PADER-SAMPLE-full-form.docx` (also provided as PDF) — a real output from our own report-generation pipeline for Bisoprolol, included to show the kind of end artifact the system produces. It's a reference for shape and tone, not a template — your report doesn't need to match its section names or wording. A couple of notes are called out directly in the document itself where relevant.

Before you start: see `DATA_USAGE_NOTICE.md` — the dataset is provided for this exercise only.

Version 0 — Required

Build a working prototype that goes from raw data to a generated report:

```
Safety data → understand/validate → analyze → AI reasoning → evidence → generate report → hu
```

Feel free to shape this differently — we care about *why* you chose your shape, not that you matched ours.

Minimum analyses

Your system should reliably produce, at minimum:

- Total cases, serious vs. non-serious
- Breakdown by age group, sex, country
- Most common reactions, and most common *serious* reactions
- Outcomes
- Any trend over time worth surfacing

Ask yourself, for each of these: does an LLM need to compute this, or does Python already give you an exact answer? A lot of candidates default to "ask the model." Consider whether the model's job here is closer to *choosing and interpreting* an analysis than *doing* it.

A worked example — grounding in practice

To make "grounding" concrete, here's the kind of chain we want to see, even in miniature:

1. Raw data (one row of 1,068):

```
safetyreportid: 12345678 patient_patientonsetage: 71 patient_patientsex: female
occurcountry: united kingdom
patient_reaction_reactionmeddrapt: Acute kidney injury
serious: serious patient_reaction_reactionoutcome: recovered/resolved
```

2. Deterministic analysis (Python, not the LLM):

```
total_cases: 1024          (unique safetyreportid – the CSV has 1,068 rows, some cases have >1
serious_cases: 1023        (99.9%)
non_serious_cases: 1
top_reactions: [Acute kidney injury: 22, Drug ineffective: 12, Cerebral haemorrhage: 7, ...]
```

3. What the model actually sees for the "Narrative Summary and Analysis" section — not the raw CSV, not a vague instruction, but an assembled, scoped packet:

```
Section: Narrative Summary and Analysis
Reporting period: [the one-year window covered by the dataset]
Approved analysis results:
  total_cases: 1024
  serious_cases: 1023 (99.9%)
```

```
non_serious_cases: 1
top_reactions: [Acute kidney injury, Drug ineffective, Cerebral haemorrhage]
Instructions: summarize only the figures above; do not infer safety
conclusions not present in the data; regulatory, neutral tone.
```

4. Output:

"During the reporting period, 1,024 cases were received, of which 1,023 (99.9%) were classified as serious. The most frequently reported reaction was Acute kidney injury."

Notice every number in that sentence traces back to step 2, not to the model's arithmetic. That traceability — and the decision of *what exactly goes into the packet in step 3* — is most of the exercise. Handing the model the raw CSV and asking it to "write a PADER" is the thing we explicitly don't want to see; it's fast to build and impossible to trust.

Show your prompt/context design

Include the actual prompts or context templates your system assembles per section. We want to see how you decided what belongs in a system instruction vs. what gets assembled dynamically per report/section, and how you kept it minimal rather than dumping everything into every call.

Generate the report

Produce one report with roughly these sections (see `PADER_Starter_Guide.md` for what each should contain): Reporting Period, Narrative Summary and Analysis, Summary Analysis of Cases, Reaction/Adverse Event Analysis, Serious Cases / 15-Day Alerts, Trends and Important Observations, History of Actions, Case Index/Listing. Markdown, HTML, DOCX, or PDF — doesn't need to look production-ready.

Human control

Somewhere in your flow, a human should be able to review before something becomes "final" — analysis results, generated sections, or both. A simple approve/flag mechanism is enough; a description of how you'd do it is also fine if you're short on time.

A Note on Scope

Version 0 is a prototype — hardcoding is fine, a single report type is fine, no auth/infra needed. But hold two things in your head while you build it: *this needs to work today*, and *this will eventually need to support report types you haven't seen yet, added mostly by changing configuration/data, not by rewriting code*. You don't need to build for that future — just don't design yourself into a corner where it's impossible.

Version 1 — Optional

If Version 0 is solid and you have time left, push on reusability. Pick whatever's interesting to you:

- **Section dependencies** — each report section declares what evidence/analyses it needs, so sections and analyses are decoupled and reusable
- **Configurable instructions** — different report types/sections carry their own generation rules instead of one global prompt
- **Reusable analyses** — the same "serious case count" logic serving multiple report types
- **Versioning** — track which dataset/analysis/prompt/model produced a given report
- **Evidence tracing** — click a sentence, see the data behind it
- **Evaluation** — some way of checking whether a generated report is actually correct, beyond eyeballing it
- Another report type, another data source, section regeneration, previous-report comparison — anything that stress-tests reusability

No time for Version 1? Submit one page instead: how would you evolve Version 0 into this? A diagram plus a short explanation is enough. We weight the thinking, not the line count.

The Real Test

The PADER is only the entry point. Now imagine we come back and ask you to support **PSUR**, **PBRER**, **DSUR**, **CSR** — each with different sections, different required data, different analyses, some overlapping with PADER and some not.

How much of your Version 0 would survive that request unmodified, if the differences were expressed as configuration and data rather than new code paths?

You don't need to answer this in writing anywhere but your README — but it's the lens we're building this whole assignment through. Don't build a PADER generator. Build a small, honest first instance of "a system that takes a reporting task, gathers the right evidence, reasons over it, and produces a controlled, traceable report" — where PADER just happens to be the first report type it knows about.

Deliverables

1. **Working Version 0** (Python prototype)
2. **One generated report** (the PADER-style output)
3. **README** — how to run it, architecture, where AI is used vs. deterministic code, models used, key prompts, design decisions, known limitations
4. **A simple architecture diagram** — components and data flow
5. **Version 1** — implementation, or the one-page design doc

See `Submission_Guide.md` for exact packaging, size limit, and where to send it.

What We're Evaluating

Area	What we're looking for
AI fundamentals	Do you understand what LLMs are and aren't good at here
Context engineering	Right information, at the right step, nothing extra
Prompt design	Clear, section-specific, reliable
Architecture	Sensible decomposition — not everything as one LLM call
Agent/tool judgment	Agents/tools used where they earn their keep, not by default
Grounding	Every claim traceable to data
Evaluation	Some notion of how you'd know the output is <i>right</i>
Generalization	Could this grow past PADER without a rewrite
Execution	It actually runs and produces a report

One more thing: more agents, more frameworks, or RAG where a lookup would do doesn't score points. We're asking "why did you build it this way," not "how many technologies does it use." The strongest submissions are the ones where every component has a clear reason to exist.